1 2 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA **Departamento de Engenharia Informática**	**Assignment #3** **Integração de Sistemas/** **Enterprise Application Integration** **2025/26** MEI, MSI **Deadline: 2026-05-15**

Notas:

A fraude denota uma grave falta de ética e constitui um comportamento não admissível num estudante do ensino superior e futuro profissional. Qualquer tentativa de fraude pode levar à reprovação na disciplina tanto do facilitador como do prevaricador.

Message Oriented Middleware (MOM) and Kafka Streams

Objectives

- Learn to write asynchronous, message-oriented, and stream-based applications.
- Learn the following Kafka-related concepts:
  - Producer
  - Consumer (and Consumer Groups)
  - Topic
  - Partition and partition offset
  - Broker
  - Zookeper (KRaft now)

Resources

Apache Kafka Quickstart: https://kafka.apache.org/quickstart/

Apache Kafka Introduction: https://kafka.apache.org

Kafka Streams: https://kafka.apache.org/documentation/streams/

Streams Developer Guide: https://kafka.apache.org/42/streams/developer-guide/

Docker Hub Kafka Image. Students should look at the overview sections for details on running standalone and multi-broker versions of Kafka:
https://hub.docker.com/r/apache/kafka

Kafka and Kafka Streams materials are also available on UC Student.

To simplify the configuration work, the professors will provide a folder with YAML and Docker files with the configuration of multiple services. Students must later configure the `lib` directory mentioned in the YAML file and optionally the `config` directory as well.

Kafka Training (doesn't count for evaluation)

1. You should start by installing Kafka (or find out where it is installed if you are using a container).

2. You may look at the reading material suggested before and follow that material to have a basic example running with a producer and consumer.

3. What happens to the messages that arrive at the topic before the subscriber makes the subscription? How do you configure the subscriber to read historical data, i.e., all messages from the beginning?

4. How do you configure different partitions inside a topic?

5. What is the point of Consumer Groups? How do they work?

6. Now, read the Kafka Streams tutorial and run the example that counts the words that go through a stream.

7. Refer to the following blog message for this and the following exercises:
   https://eai-course.blogspot.com/2018/11/playing-with-kafka-streams.html

   Use Kafka Streams to count the events that happened for a given key. Display the result as 'k->v' in the final stream.

8. How do you change the type of data of the keys and values that you send?

9. How do you switch the key and value of a stream?

10. Now sum all the values that showed up for each key.

11. Now, control the time considered in the stream to be 1 minute.

12. How could you send complex data instead of primitive types?

13. Imagine that you have two streams, *e.g.*, debit and credit. How do you create a third stream with the current balance?

14. Use the Kafka Connect application to periodically fetch data from a database table (source). You can find help for this operation in the following blog message: https://eai-course.blogspot.com/2019/11/how-to-configure-kafka-connectors.html

15. Now do the inverse operation: send from a topic to a database table (sink).

16. Data in Kafka Topics are streams of bytes. This means that data needs to be serialized and de-serialized into and from appropriate formats for consumption. Kafka supports serialization and de-serialization (serdes) of primitive types (String, Long, etc), custom serializers and common serialization formats, namely, AVRO, Protobuf & JSON. Using the below guides try and serialize data to and from Kafka topics with an Avro, Protobuf or JSON formatter WITHOUT Schema Registry.
https://docs.confluent.io/platform/current/streams/developer-guide/datatypes.html

17. Kafka supports several delivery semantics: "At least once", "At most once" and "Exactly one". There are a series of pre-requisites and caveats, especially for the "exactly once" semantics (e.g. idempotent producer, isolation level of the consumer), namely the way Kafka deals with the consumer offsets. Try and implement a highly constrained scenario (one consumer, one producer) where "exactly once" semantics can occur.
https://docs.confluent.io/kafka/design/delivery-semantics.html

Project Description (for evaluation)

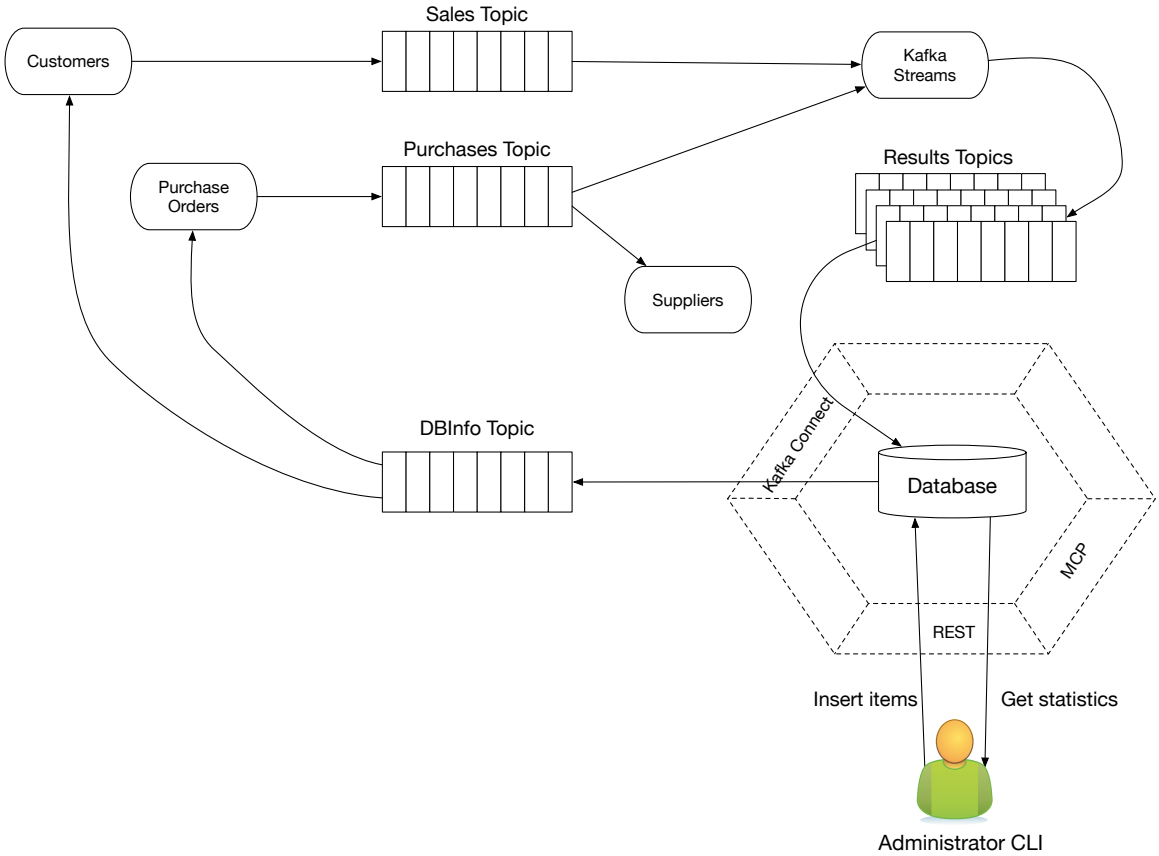

Figure 1 - Overview of the Shop

Students should define their own application, e.g., by evolving the work from the previous assignment, write their own requirements, and implement their solution. The application and requirements should have a level of complexity that is similar to the example that follows. Students should discuss their option in detail with the professors.

Example Application

Refer to Figure 1, summarizing a shop in Kafka. The shop has the following Kafka topics:

- **DBInfo**, where one or more source connectors write a list of countries and a list of items for sale from the database.
- **Purchases** topic, where a process simulating purchases to suppliers keeps writing orders of items listed in the DBInfo topic. Each purchase item includes a price and a number of units. The item to purchase, its price, and the units can be randomly determined.
- **Sales** topic, where a process simulating customers keeps writing. Each sale has an item, a price, a number of units and a country of origin. All of these are randomly generated, but the items and countries must be listed in the DBInfo topic. In this example, one does not need to be concerned about whether the stock for each item is positive or if the shop is profitable.

- One or more applications using Kafka streams will listen to these topics and compute a number of metrics, as enumerated in the requirements, including revenues, expenses, profit, etc. This or these applications will then write their computations to another topic, the **Results** topics, from where a Kafka connector will write the results back to the database.

A server-side application will read data from the database and expose the data via REST. The final piece of the application is a command line application that allows the administrator to enter items, countries and read statistics from the REST services. No authentication is necessary.

Components to Develop

In their own case, students need to develop applications that are similar to the following ones of Figure 1: Customers, Purchase Orders, Kafka Streams. These are stand-alone applications. Students must also configure Kafka connectors to automatically extract and write data from/to the database via the Results and DBInfo topic.

Students must determine the precise definition of the communication format. Clean solutions that serialize complex data structures as JSON strings or byte arrays are encouraged. This includes the use of standard Kafka Serializer/De-serializer constructs.

Requirements

Students should write their own requriments, which should include challenges, to be solved with Kafka Streams, that are of similar complexity to the following actions:

1. Add countries to the database. To simplify countries cannot be deleted and optionally not changed.
2. List countries from the database.
3. Add items for sale in the shop to the database. Again, these cannot be deleted, but may be changed if students wish.
4. List items from the database.
5. Get the revenue per item.
   - This is solved with a `reduce()`.
6. Get the expenses per item.
   - This is solved with a `reduce()`.
7. Get the profit per item.
   - This requires a a `join()`.
8. Get the total revenues.
   - One possible solution is to use a `groupBy()` instead of `grupByKey()`
9. Get the total expenses.
   - Similar to the previous case.
10. Get the total profit.
    - This requires a `join()`.
11. Get the average amount spent in each purchase (separated by item).
    - This requires an `aggregate()`.

12. Get the average amount spent in each purchase (aggregated for all items).
    - `groupBy()` and `aggregate()`.
13. Get the item with the highest profit of all (only one if there is a tie).
    - This could require a `reduce()` or `aggregate()` plus some data structure to keep historical data.
14. Get the total revenue in the last hour[1].
    - This should use a tumbling time window.
15. Get the total expenses in the last hour.
    - This should use a tumbling time window.
16. Get the total profits in the last hour.
    - This should use a tumbling time window.
17. Get the name of the country with the highest sales per item. Include the value of such sales.
    - This case is slightly more complicated than finding the item with the highest profit.

Students should include means in their application to enable a fast verification of the results they are displaying. Solutions should maximize the computations they do with Kafka streams and reduce the database queries to their simplest possible form (for example, students should use Kafka Stream to compute profits, instead of computing them on the database).

Students should configure Kafka with multiple brokers to enable Fault-Tolerance and should demonstrate the fault-tolerance abilities of their work.

Final Delivery

- The work should be done in groups of **2 students**. Both group members must be fully aware of all the details as their defense may occur separately. Work together!
- Students should prepare and submit a pdf presentation for 5 minutes at Inforestudante. Students must also submit the source code for discussion.
- The defense will be based on this presentation.
- Grades will be based on the quality of the presentation and defense.
- No report is necessary.

Good Work!

[1] Students may change this interval freely for testing and demonstration purposes.